UNISOLE AI CAMPUS PROGRAM

GROUP 1 — COMPUTER SCIENCE & IT

Combined Pathway: Machine Learning Engineering + Full Stack Web Development

Target students: BCA • MCA • B.Sc. CS/IT • PGDCA • B.Tech CSE/IT • Related disciplines

Format: Weekday technical track (can be paired with the weekend AI Entrepreneurship & Innovation track)

Structure: 14 core modules across two integrated tracks, converging into one full-stack, AI-powered capstone

---

Overview

This combined pathway is for students who don't want to choose between "AI/ML" and "web development" — because in the real world, the most valuable engineers are the ones who can do both. A model that can't be served to real users is just a research exercise; a web application with no intelligence built in is just a form. This pathway trains students to build the entire stack: the data pipeline and model on one side, the interface, backend, and database on the other, and — critically — the integration layer that connects them into one working AI-powered product.

Why this combination matters: Companies increasingly don't hire separately for "the ML person" and "the web person" on small teams — they hire engineers who can own a feature end to end. This pathway is structured so a student leaves able to say, credibly, "I built the model, the API, the app, and shipped it."

How the two tracks fit together: Part A (Modules 1–7) builds the AI/ML engineering skillset — from Python fundamentals through to Generative AI and RAG. Part B (Modules 8–14) builds the full-stack web development skillset — from HTML/CSS through to AI-powered web applications. The two tracks share a common language (APIs, Git, deployment, Docker) so students see clearly how a trained model becomes a feature inside a real product, rather than treating the two as separate worlds.

PART A — MACHINE LEARNING ENGINEERING

Duration: ~3 months of the combined program

Recommended level: Intermediate / students with basic Python exposure

This track takes a student from writing basic scripts to shipping a working AI product — sequenced so each module unlocks the next: programming fundamentals feed into data handling, data handling feeds into classical ML, classical ML feeds into deep learning and generative AI.

Module 1 — Python for AI Engineering

This module builds the coding foundation every later module depends on. Rather than teaching Python as an isolated language, it is taught as an engineering tool — how professionals structure, reuse, and share code.

- Python programming fundamentals

- Functions, modules and packages
- Object-oriented programming
- Exception handling
- File and data handling
- Virtual environments
- Package management
- Writing reusable code
- Git/GitHub workflow

Practical: Build a small Python application and publish it on GitHub.

Module 2 — Data Engineering Foundations

Before any model can be trained, data has to be sourced, cleaned, and organized. This module teaches students how real companies move and store data — from raw files to queryable warehouses.

- Structured vs unstructured data
- CSV, JSON and Parquet
- NumPy, Pandas
- Data cleaning and validation
- Exploratory data analysis
- SQL fundamentals and relational databases
- OLTP vs OLAP
- ETL vs ELT
- Batch vs streaming data
- DuckDB
- Apache Kafka fundamentals

Practical: Data Source → ETL → Parquet → DuckDB → Analytics / ML

Module 3 — Machine Learning

Students learn how to frame a business problem as a learning problem, choose the right algorithm, and measure whether the resulting model is actually good enough to trust.

- ML workflow
- Supervised learning — Regression, Classification
- Unsupervised learning — Clustering
- Feature engineering
- Train/validation/test split, Cross-validation
- Model evaluation — Accuracy, Precision, Recall, F1-score, Confusion matrix
- Model selection

Module 4 — Deep Learning & Modern AI

Students move into the architectures that power today's AI systems — from image-recognition networks to the transformer models behind large language models.

- Neural networks, Forward/backpropagation concepts
- CNN fundamentals

- Sequence models overview
- Transfer learning, Model fine-tuning concepts
- Introduction to Transformers
- Embeddings
- LLM fundamentals

Module 5 — Building AI Applications

A model sitting in a notebook is not useful until it can be called by real software. This module — and this is the direct bridge into Part B — teaches students to wrap a trained model in a production-style API.

- Model inference
- REST APIs
- FastAPI
- Request/response validation, Pydantic
- Connecting models to applications
- Authentication fundamentals
- Error handling

Module 6 — MLOps & Deployment

The operational discipline that keeps an AI system reliable after launch, not just on the day it's built.

- Model serialization
- Experiment tracking, Model/version management
- Docker, Containerization
- CI/CD fundamentals
- Cloud deployment concepts
- Monitoring, Data drift, Model drift
- Retraining pipelines

Module 7 — Generative AI & RAG

Students learn to build applications on top of large language models, including Retrieval-Augmented Generation (RAG) systems.

- LLM architecture overview
- Prompt engineering
- Embeddings, Vector databases
- RAG architecture
- Document ingestion, Chunking
- Retrieval, LLM response generation
- AI application evaluation
- Agent fundamentals

Part A output: A trained model, wrapped in a working API, containerized and deployable — this becomes the "intelligence layer" plugged into the Part B web application later in the combined capstone.

PART B — FULL STACK WEB DEVELOPMENT

This track moves from front-end fundamentals to backend systems to databases, then layers in AI features — using the exact model/API built in Part A as the AI feature powering the final application.

Module 8 — Web Foundations

The building blocks of every website: how a browser renders content, how pages respond to user interaction, and how a page talks to a server.

- HTML, Semantic HTML
- CSS, Responsive design
- JavaScript fundamentals
- DOM, Events
- Async JavaScript
- Fetch/API consumption

Module 9 — Modern Frontend

Students learn React, the framework used by most modern web companies, to build interactive, component-based user interfaces.

- React
- Components, Props, State, Hooks
- Forms
- Routing
- API integration
- Vite
- Frontend project structure

Module 10 — Backend Development

The server side of the application — how requests are received, processed, secured, and answered.

- Node.js, Express
- REST APIs
- Middleware
- Authentication, Authorization
- Validation
- Error handling
- API documentation

Module 11 — Databases

Every application needs a place to persist data. This module covers MongoDB and how to design and query it correctly.

- Database fundamentals
- MongoDB
- CRUD
- Schema design

- Indexing
- Aggregation
- Connecting backend with database

Module 12 — Software Engineering

The professional practices that separate a hobby project from a maintainable, team-ready codebase.

- Git, GitHub, Branching, Pull requests
- Code organization
- Environment variables
- Debugging
- Testing fundamentals

Module 13 — Deployment

Turning a local project into a live, publicly accessible application — using the same Docker/CI/CD foundations built in Part A, Module 6.

- Production builds
- Hosting
- Environment configuration
- Docker fundamentals
- CI/CD concepts
- Monitoring basics

Module 14 — AI-Powered Web Applications

The convergence module: students connect the Part A model/API directly into the Part B web application.

- Connecting LLM APIs
- AI features inside web applications
- RAG-based applications
- AI assistants, Document Q&A;
- AI-powered dashboards

Part B output: A complete frontend + backend + database application, ready to be connected to the AI/ML layer from Part A.

COMBINED CAPSTONE

Students build one unified, end-to-end AI-powered product that fuses both tracks into a single system:

Data → ML Model → API (Part A)

+

Frontend → Backend → Database (Part B)

→

One deployed AI-powered web application

This is the pathway's signature outcome: not two separate projects, but a single product where a real trained model, served through a real API, powers a real feature inside a real web application — end-to-end, built and deployed by one student.

Example capstone shapes: An AI-powered document Q&A; web app; a web platform with a built-in recommendation or prediction feature; a dashboard application with an embedded ML-driven insight engine; a full-stack RAG-based assistant with user accounts and a persistent database.

Tools & technologies used across this combined pathway: Python, Pandas, NumPy, SQL, DuckDB, FastAPI, Pydantic; HTML, CSS, JavaScript, React, Vite, Node.js, Express, MongoDB; Git/GitHub, Docker, CI/CD; LLM APIs, vector databases.

Career relevance: Full Stack AI Engineer, AI Product Engineer, ML Engineer with product ownership, Full Stack Developer with AI specialization — roles that increasingly sit at the intersection of both skillsets rather than in either silo alone.

Student outputs: GitHub repository (frontend + backend + ML), a deployed full-stack application with a working AI feature, API documentation, technical documentation, project presentation, resume project entry.

Industry-Readiness Layer *(applies alongside this pathway)*

Professional Digital Profile: Professional email, LinkedIn, Resume, Portfolio, Project documentation, Professional communication.

GitHub / Evidence: Git, GitHub, README, Repository structure, Version control — presented across both the ML and web codebases.

Interview Preparation: Resume walkthrough, Tell-me-about-yourself, Project explanation, Behavioural questions, Technical/domain questions, Mock interview, Feedback.

Industry Interaction: Weekly/periodic expert sessions, Career sessions, Industry case studies, How companies recruit, How to approach professionals, Networking etiquette.

Project Defence: Every student must be able to answer — What problem did you solve? Why did you choose this approach? What did you build? What did you learn? What are the limitations? What would you improve? (For this combined pathway, students should be ready to defend choices on both the model side and the application side.)

Final Capstone Framework

1. **Problem** — Identify a real-world problem
2. **Research** — Understand the domain and existing solutions
3. **Data/Information** — Collect and organize relevant information
4. **Build** — Create the ML model, the API, and the full-stack application
5. **Validate** — Test the model's performance and the application's functionality
6. **Document** — Report, Repository/portfolio, Documentation, Presentation
7. **Defend** — Present the project before an evaluator/mentor

What This Student Should Have at the End

A single deployed AI-powered full-stack product + GitHub repository (ML + web) + API + technical documentation + resume project entry

Training and assessment are program components. Internship, advanced projects, hackathons, research engagement and other opportunities are performance/availability-based and subject to applicable selection procedures.